FORM-TL-01
Notas de presentación: Vecindario

APUNTE PAZMIÑO WASHINGTON GERMAN
LOOR SUAREZ JORDAN JOSUE
MACIAS VASQUEZ CESAR ABRAHAN

Periodo académico 2026–2027

1. Contexto del escenario

Vecindario es una red social ficticia orientada a comunidades de barrio en ciudades intermedias de Ecuador.

La plataforma permite publicar avisos, seguir a vecinos y negocios locales, reaccionar, comentar y mantener conversaciones mediante mensajes directos.

2. Arquitectura propuesta

3. Mecanismos de comunicación

4. Operaciones principales del diagrama

4.1. Mensajería directa, presencia y moderación

Recorrido de un mensaje directo. Al abrir la aplicación, el cliente (CLI) establece un canal WebSocket seguro (WSS sobre TCP) con el balanceador, que lo asigna a una de las instancias de MSG (S4). El canal queda abierto, por lo que los mensajes se envían y se reciben en ambos sentidos sin crear una conexión nueva cada vez. Cuando un usuario envía un mensaje, MSG le asigna un `sequence_no` creciente dentro de su conversación y lo guarda en PostgreSQL con `conversation_id`, `message_id`, remitente, destinatario, contenido y *timestamp* (S6). Solo después de que PostgreSQL confirma la escritura, MSG envía el ACK al remitente y entrega el mensaje al destinatario si está conectado.

Presencia en línea. Mientras el canal está abierto, el cliente envía periódicamente un *heartbeat* por el mismo canal WSS (S5). Con cada uno, MSG renueva en Redis la clave de presencia del usuario, que tiene un tiempo de expiración (TTL) (S7). Si los *heartbeats* dejan de llegar, la clave expira y el usuario se muestra como desconectado, sin necesidad de un aviso explícito.

Denuncia y ocultamiento. Cuando un usuario denuncia un aviso, el cliente envía a MOD el `id_aviso`, el `id_usuario` y el motivo (S8), y MOD confirma la recepción. MOD cuenta únicamente las denuncias válidas de usuarios distintos. Al llegar a tres, solicita a PUB ocultar el aviso (S9); PUB cambia su estado a oculto y publica en Kafka el evento `PublicaciónOcultada` (A4), que MUR consume para retirar o invalidar el aviso en los muros.

ID	Origen → destino	Tipo	Sincronía	Protocolo y middleware
S4	CLI ↔ balanceador ↔ MSG	Socket persistente	Bidireccional	WSS/TCP
S5	CLI ↔ MSG	*Heartbeat*	Bidireccional	WSS/TCP
S6	MSG ↔ PostgreSQL	Escritura y confirmación	Síncrona (antes del ACK)	Protocolo PostgreSQL/TCP
S7	MSG ↔ Redis	Presencia con TTL	Síncrona breve	RESP/TCP
S8	CLI → MOD	Petición-respuesta	Síncrona	HTTPS/TCP, REST API
S9	MOD → PUB	RPC	Síncrona	HTTP/2, gRPC
A4	PUB → Kafka → MUR	Mensajería	Asíncrona	TCP, Kafka

Mecanismos y justificación.

- **WSS sobre TCP, bidireccional y persistente (S4, S5).** Un canal abierto permite entregar el mensaje apenas llega, sin el costo de abrir una conexión por cada envío, y sirve también para el *heartbeat* de presencia. TCP garantiza entrega ordenada y confiable; WSS cifra el tráfico.
- **PostgreSQL síncrono antes del ACK (S6).** Un mensaje no se confirma hasta estar guardado, para no confirmar algo que luego pueda perderse.
- **Redis con TTL (S7).** La presencia es un dato temporal que cambia constantemente y no necesita sobrevivir a un reinicio, por lo que se guarda en memoria con expiración.
- **REST y gRPC en la moderación (S8, S9) y Kafka en la invalidación (A4).** La denuncia necesita confirmación inmediata; la orden de ocultar es una llamada directa entre servicios internos; y el retiro del aviso en los muros puede hacerse de forma asíncrona.

R2: mensajes en menos de 500 ms y en orden. El canal WSS ya está abierto, por lo que la entrega a un destinatario conectado no depende de establecer conexiones nuevas. El `sequence_no` creciente por conversación permite mostrar los mensajes en el orden en que fueron enviados, y el `message_id` único permite descartar un mensaje repetido si hubo un reintento. Guardar antes del ACK evita confirmar un mensaje que todavía no está persistido.

R5: tres denuncias y ocultamiento en menos de un minuto. MOD cuenta tres denuncias válidas de usuarios distintos y solicita a PUB ocultar el aviso (S9). PUB cambia el estado y publica `PublicaciónOcultada` (A4); MUR invalida el aviso en los muros de los seguidores. Como la secuencia es corta y solo la última parte es asíncrona, se espera cumplir el límite de un minuto.

R6: disponibilidad de 99,5 % en la mensajería. Se aspira a esa meta con varias instancias de MSG detrás del balanceador, *health checks* que retiran las instancias que no responden y persistencia de los mensajes en PostgreSQL. El muro está desacoplado mediante Kafka, de modo que su degradación no bloquea la mensajería.

F1: el destinatario está desconectado. El mensaje ya se guardó en PostgreSQL antes del ACK, por lo que queda pendiente aunque el destinatario no esté en línea. Cuando vuelve a conectarse, el cliente pide los mensajes posteriores a su último ACK y MSG se los entrega.

F3: MSG se reinicia con conversaciones abiertas. Los clientes se reconectan al balanceador, que los asigna a una instancia disponible. Cada cliente recupera desde la última secuencia confirmada, sin depender de memoria volátil, y el `message_id` permite descartar lo que ya se hubiera procesado.

5. Protocolos y middleware

6. Requisitos y situaciones de fallo

7. Preguntas críticas recibidas y respuestas

1. **Pregunta recibida:**
   Equipo que la formuló:
   Respuesta del equipo:

8. Preguntas críticas formuladas a otros equipos

1. **Equipo al que se dirigió:**
   Pregunta formulada:

9. Retroalimentación y ajustes